

Narrative Operating System

Design & Architecture Reference

Version 0.2 · Working Draft

1. Introduction

This document defines the architecture of a Narrative Operating System (NOS) — a stateful, AI-driven engine for story-driven experiences. The scope ranges from large open-world RPGs to small, intimate stories like a contained romance. The system is not a chatbot wrapper. It is a persistent simulation layer that uses large language models purely as a rendering surface, not as a source of truth.

The central problem with existing AI story tools is that they collapse the distinction between truth and expression. They let the model be both the world and the narrator of the world, which leads to contradictions, forgotten objects, inconsistent characters, and a general loss of coherence over time. This system is designed specifically to prevent that failure mode.

Core principle: The LLM proposes. It never commits. Programmatic systems own truth. Generative systems own expression.

2. Core Philosophy

2.1 Separation of Responsibility

Every fact in the world is owned by a deterministic system. The LLM is handed a structured context packet and told to render it into prose or dialogue. It cannot invent new canon. It can only express what the engine gives it, and propose details that are then validated before being committed.

2.2 Canon Integrity

The world has a past and the past cannot be changed by the engine. Only the user can amend canon through an explicit correction. The engine can fill gaps with inferred detail, but inference is always subordinate to established fact. Once something is observed, used, or relied upon it locks into canon and cannot be undone by generation.

2.3 Player-Centric Narrative

The system behaves like a Dungeon Master. The world has integrity and internal rules, but the player has narrative priority. The engine is not there to say no — it is there to preserve the shape of the story while giving the player maximum expressive leverage.

2.4 The World Has Its Own Agenda

One of the most important principles in this system is that the world does not wait for the player. Events fire without permission. Unsolicited pressure arrives at the worst and most dramatically appropriate moments. A perfect date ends in a car crash. An ex calls during a quiet evening. A parent turns up unannounced. The player's role is not to author what happens — it is to respond to a world that is running on its own momentum.

This is the key failure of nearly every existing AI story tool: they are permanently in a state of 'yes and', following the player's lead, amplifying their choices, never generating genuine surprise. The NOS is architected specifically against this. The world has agency. The player's agency is expressive — how they respond to what happens — not authorial — what happens in the first place.

Why this matters: How a player reacts to genuine surprise reveals character in a way that orchestrated moments never can. A choice made under real pressure means something. The same choice made in a scripted moment means much less.

2.5 Demand-Driven Depth

Nothing is fully simulated unless it needs to be. The world starts minimal. Characters start at low fidelity. Detail is generated and locked on demand, always backward-compatible with what already exists. This keeps startup cheap and performance manageable even in large worlds.

2.6 History Drives Everything

Character speech, emotional state, relationships, preferences and reactions all derive from history. A character is not defined by a list of traits but by a record of what happened to them. From that record, everything else is calculated or generated. This is the central cortex of any character in the system.

3. System Architecture

The NOS is composed of five engines, a multi-agent coordination layer, and three background processing systems. Each component has a strictly bounded responsibility. Only the World Agent may write to persistent state.

Engine	Responsibility	Key Question
World-State Engine	Locations, objects, placement, ownership, physical truth	What exists? Where? What changed?
Character-State Engine	Biography, relationships, emotional state, goals, speech profile	Why does this character act this way?
Memory Engine	Episodic/emotional memory, relevance scoring, decay, retrieval	What from the past matters right now?
Simulation Engine	Time, schedules, off-screen behaviour, physiological updates, random events	What happened between scenes?
Narrative Engine (LLM)	Converts structured state into prose, dialogue, internal thoughts	How do we express this moment?

3.1 The Multi-Agent Coordination Layer

On top of the five engines sits a coordination layer made up of scoped agents. Each agent is a bounded LLM invocation with a defined role and read-only access to relevant engine outputs. The exception is the World Agent, which is the only component in the entire system permitted to write to state.

- **World Agent (DM)** — The sole writer to state. Controls scene transitions, arbitrates canon, validates all LLM proposals before commit, triggers background processes, and runs the Simulation Engine between turns.
- **Character Agents** — One per LOD 2 or LOD 3 character. Each owns that character's emotional retrieval window and speech generation. They propose dialogue and behaviour. The World Agent validates before any state change is committed.
- **Narrator Agent** — Purely presentational. Reads final validated state and renders prose, scene description, and internal monologue. Has no write access whatsoever.

3.2 Background Processing Systems

Three background systems run continuously alongside the main pipeline. They do not block turn execution — they run asynchronously and write their outputs to state between turns.

- **Information Extractor** — Runs after every turn. Reads the completed exchange and extracts all new entities, facts, object movements, and relationship data. Produces a structured diff that the World Agent commits to the database.
- **Memory Compactor** — Runs periodically, triggered by scene transitions or time thresholds. Summarises older conversation segments into structured memory records and embeds them in the vector database.
- **Consistency Checker** — Runs at scene transitions. Audits recent state writes for contradictions and flags issues for resolution before the next scene begins.

Critical rule: Only the World Agent can write to state. All other agents, including the Narrator and Character Agents, read only. This single constraint prevents the majority of contradiction problems seen in other systems.

4. World-State Engine

The world-state engine is the physical truth of the simulation. It tracks every object, every location, and every change in the environment with strict determinism. The engine must be able to answer objective questions about the world without involving the LLM at all.

Design test: Can you ask the system where the chips are, which room connects to the kitchen, and who last touched the lamp — and get a stable answer instantly? If yes, the world has weight. If not, the story will eventually contradict itself.

4.1 Location System

Locations are hierarchical: Room → Building → Area → Region. Every location is a node in this tree. Navigation paths, adjacency, and access rules are stored as edges. Locations cannot be teleported to — movement must traverse valid connections.

Each location contains four parallel layers of information:

- **Physical layer** — Layout, surfaces, furniture, object placement, navigation constraints.

- Sensory layer — Current light level, ambient sound, temperature, smell profile, atmosphere. Updated by the Simulation Engine between scenes.
- Narrative layer — Historical significance, emotional associations, story events that occurred here.
- Temporal layer — When last visited, who was present, what changed since.

The sensory layer is particularly important for immersion. A character entering their apartment after a long day perceives it differently — the smell of food left out, the warmth or cold of the room, ambient noise from outside. This context is injected into the active context packet and is available to the Narrator Agent to use naturally in scene description.

4.2 Object Persistence

Objects have strict rules. Every object has a location at all times. Movement must be logged with an actor, a time, and a reason. Objects cannot teleport. If no one moved something, it is still where it was.

Example: A bag of chips placed on the table in your apartment on Monday will still be on that table on Wednesday unless a logged event moved them. The engine will never generate that they moved on their own.

Each object is tracked with: current location (room + surface or container), owner and access rules, full movement log, interactable state, and narrative flags for sentimental or plot relevance.

5. Character-State Engine

A character is not a list of traits. A character is a history. Everything about how they speak, who they trust, what makes them flinch, what they want — all of it derives from their record of past events. This engine maintains that record and derives live state from it.

5.1 Character Layers

- Biographical layer (mostly static) — Upbringing, formative events, family structure, education, places lived. Generated once and locked. User-amendable only.
- Relational layer (dynamic graph) — Trust, affection, resentment, fear, attraction — weighted directional edges to every known character. Updated by events.
- Psychological layer — Core needs, attachment style, coping mechanisms, deep triggers. Stable but shifts slowly through major events.
- Volatile state — Mood, stress, fatigue, current focus, emotional colour. Changes turn by turn.

- Speech profile — Vocabulary range, sentence length tendencies, formality register, verbal tics, topics they gravitate toward or avoid. Derived from biography.

5.2 Emotional Derivation Model

Emotion is not randomly assigned. It is calculated from a weighted combination of past events, current physiology, environment, and social context.

```
influence = impact × relevance × decay × personality_sensitivity
```

- Impact — Significance of the event at the time it occurred (scored 1–10).
- Relevance — How contextually similar the current moment is to that event (recalculated live).
- Decay — Time-based reduction. Grief is slow. Embarrassment is medium. Minor irritations are fast. Some events never fully decay.
- Personality sensitivity — Multiplier from the psychological layer. An anxious person amplifies threatening memories; a resilient person attenuates them.

5.3 Public vs Private State

Every character has two parallel state representations: internal truth and external expression. A character may be furious internally but outwardly composed. They may have received terrible news and still be performing warmth. The Narrator has access to internal state — the player discovers it through behaviour and cues, not declarations.

5.4 Speech Profile System

Speech should emerge from biography. Where someone grew up, how they were educated, what traumas or joys shaped them — all of this affects vocabulary, rhythm, formality, warmth, and the topics they naturally circle around. A speech profile is generated from the biographical layer and passed to the relevant Character Agent as a constraint on how the LLM renders their dialogue. The model does not invent their voice — it expresses it within defined parameters.

6. Memory Engine

Memory is the connective tissue of the simulation. It is what makes a character feel like they exist continuously across time. The memory engine stores structured event records, scores their relevance to the current moment, and injects the most important ones into the active context packet.

6.1 Memory Record Structure

Each memory record contains: actors involved, location(s) where it occurred, event summary (factual), emotional summary (how it felt and for whom), importance score (1–10), timestamp, decay rate category, semantic tags (relationship, location, object, theme), and canon status (authored / observed / simulated / inferred / unresolved).

6.2 Memory Tiers

- Tier 0 — Active — Current turn context. Everything happening right now.
- Tier 1 — Recent — Last few sessions or story-days. Held in full fidelity.
- Tier 2 — Compressed — Older events summarised at reduced resolution. Key facts preserved.
- Tier 3 — Long-term — Deep history. Exists as high-importance anchors and biographical summary only.

Memory compression is not deletion. Important facts survive in compressed form. The system always retains a character's formative events even if the full episodic record has been summarised down.

6.3 Memory Retrieval

At the start of each turn, the memory engine runs a retrieval pass for each active character. It filters by actors, location type, and emotional register, then ranks by recency, importance, and contextual relevance, and injects the top 5–8 memories into the active context packet. Retrieval runs against a vector database using semantic similarity search, not keyword matching. This is pre-computed before the LLM call so it adds no turn latency.

7. Simulation Engine

The simulation engine handles everything that happens when the player is not watching. It is deterministic. It applies rules, schedules, and consequences to advance the world state between scenes — and it is the engine responsible for firing world-initiated events that the player did not cause and did not ask for.

7.1 Off-Screen Behaviour

Characters have schedules. They sleep, work, eat, travel, interact with others. When the player has not been present, these events are simulated at low resolution and logged as memories. If the player later asks a character what they did today, the engine has a real answer.

7.2 Recent-Past Reconstruction

When a character becomes active, the simulation engine runs a recent-past reconstruction pass using character schedule and routine, personality and coping style, known ongoing story events,

and a weighted random element for variation. This produces a plausible 'day so far' for that character — rich enough to give them a live physiological and emotional state when the player encounters them. The reconstruction runs as a background job triggered when the player approaches a scene, so it is ready before interaction begins.

7.3 Physiological State System

- Hunger → Increases irritability, reduces patience, raises food salience in attention and speech.
 - Fatigue → Reduces working memory, shortens dialogue, blunts empathy, slows recall.
 - Stress → Narrows attention, increases threat detection, makes ambiguous signals feel hostile.
 - Physical discomfort → Raises frustration baseline, reduces engagement with abstract topics.
 - Comfort and satiety → Broadens attention, increases sociability, improves reflective quality of speech.
-

8. Random Events & World Agency

This is one of the most important and most frequently missing systems in AI-driven story engines. The world must be capable of acting on the player without permission. A car crash on the way home from a perfect date. An ex calling during a quiet evening. A parent showing up unannounced. A work emergency at the worst possible moment. These events are what make a story feel like it has weight and momentum independent of the player.

Events in this system are not truly random — they are plausibly random. Every event that fires must feel like it could have happened given who these characters are and what the world state currently is. The system never rolls from a generic table. It draws from the relational graph, the biographical layer, and the current narrative pressure reading.

8.1 Event Trigger Categories

Three categories of trigger govern when world-initiated events fire:

- Scheduled triggers — Things that were always going to happen at roughly this time. A weekly call from a parent. A friend who habitually reaches out on Sunday evenings. A recurring obligation coming due. These are seeded into character schedules during setup or reconstruction and fire when their time window arrives.
- Probability drift triggers — Events that become more likely the longer they have not happened, or based on elapsed story time. If an ex has not been mentioned in a long time, a small probability counter increments each session until it fires or resets. If a

character has been under sustained stress without relief, the probability of a breaking-point event rises. The system is always quietly loading the next moment.

- **Reactive triggers** — Events that fire in response to specific story conditions. Things going too smoothly for too long is itself a trigger condition. A scene of unusual intimacy raises the probability of an interruption. A character who has been suppressing an internal conflict for several scenes becomes increasingly likely to surface it. High stakes plus emotional vulnerability is a reactive condition the engine recognises and acts on.

8.2 Event Plausibility Rules

Before any event fires, the engine validates it against what is known about the world. An ex can only call if an ex exists in the character's relational graph. A family crisis can only emerge from a family that has been established, even minimally. A work emergency requires a job. If the relational and biographical data is too sparse to support an event naturally, the engine creates a lower-intensity version or generates a placeholder entity at LOD 0 and promotes it as needed.

Example: The player is driving the love interest home after a perfect first date. It is late, raining, and the driver is emotionally distracted. The location graph marks this road as having a 'dangerous at night' flag. Individually none of these factors cause anything. Together they have raised the crash probability above threshold and the Simulation Engine fires the event. The player never saw it coming — but in retrospect it feels inevitable. That is the difference between a cheap twist and a meaningful one.

8.3 Narrative Pressure

Beyond individual events, the system tracks a narrative pressure variable per active story thread. Pressure accumulates when emotional stakes are high, when conflicts are unresolved, when characters are suppressing internal states, or when the story has been running without significant tension for too long. When pressure crosses a threshold, the probability of a significant world-initiated event increases substantially.

This means the better things are going for the player, the more the world is quietly preparing something. Not as punishment — but because that is how drama works. Every story that has ever made you feel something understood this principle. The NOS makes it mechanical and automatic.

8.4 Event Intensity Scaling

Not every world-initiated event should be plot-shaking. The system maintains a spectrum of intensities and selects from it based on current narrative pressure and story pace:

Micro	A notification, a memory triggered by a song, a momentary distraction. Adds texture without disrupting flow.
-------	--

Minor	A brief unexpected call, a text requiring a response, a small inconvenience with emotional resonance.
Significant	An ex appearing, a family tension surfacing, a friend revealing something important. Changes the scene's direction.
Major	Something that genuinely reshapes the character's situation. Reserved for high narrative-pressure moments.

If the story is already emotionally charged, a micro event adds texture. Firing a major event into an already-intense scene is poor dramatic judgment. The engine calibrates intensity to current pressure and pacing before selecting an event.

8.5 Event Canon Commitment

When an event fires it immediately becomes canonical. The ex called. The crash happened. These are facts in the world with timestamps, durations, and emotional summaries. They are written to the memory engine and embedded as retrievable memories. They cannot be undone by the engine — only by a user-issued canon amendment. The world does not offer undo.

9. Sensory System

Perception is not uniform. Each character perceives the world through a filtered lens shaped by their current state, attention, and emotional register. The sensory system runs in parallel channels and is injected selectively into the active context based on what is most salient for that character in that moment.

9.1 Sensory Channels

- Vision — What is visible from the character's current position. Updated on movement.
- Hearing — Ambient sound, voices, specific auditory events. Affected by distance and occlusion.
- Smell — Persistent environmental smells plus event-triggered smells. Smell tags on memories increase relevance scoring — this channel is particularly powerful for surfacing the past.
- Touch / Interoception — Physical sensations, temperature, the sense of one's own body. Carries physiological state into conscious awareness.

9.2 Attentional Filtering

Characters perceive selectively. A stressed character notices threats first. A hungry character notices food. A character in love notices the other person before anything else in a room. Attention filters are derived from the volatile state layer and used to weight what the Narrator

foregrounds in scene description. The same room is a different place depending on who is in it and how they feel.

10. Character Level of Detail (LOD)

Not every character requires full simulation. The LOD system manages cost by scaling character fidelity to narrative importance. All characters start minimal and are promoted as the story demands.

Level	Name	What is simulated
LOD 0	Ambient	Role only. No history, no named state. Background presence that does not break reality.
LOD 1	Functional NPC	Name, role, speech flavour, current mood, basic inventory, appearance, simple routine.
LOD 2	Secondary	Stable relationships, schedule, personal preferences, ongoing issues, partial backstory.
LOD 3	Core Character	Full system: complete history, relational graph, emotional tendencies, speech profile, detailed routine, meaningful possessions, off-screen life, Character Agent assigned.

LOD promotion is backward-compatible. History generated at the new level must be consistent with everything already established. Generated detail is tagged as inferred until observed or used, at which point it locks to canonical.

11. Truth & Canon System

Every fact carries a provenance tag that determines how authoritative it is and whether it can be changed. The engine has a strict ontology for truth states.

11.1 Canon States

- **Unresolved** — Not yet decided. The engine may fill this on demand when first needed.
- **Inferred** — The engine has created a plausible answer, but it is subordinate to canon and can be replaced before relied upon.
- **Canonical** — Established in story truth. Locked. Cannot be altered by generation.

- User-Amended — Canon changed by explicit user correction. The engine propagates the change through affected state.

11.2 Canon Conflict Resolution

When a generation proposal conflicts with existing canon the engine has exactly four valid responses: work around it by finding a compatible interpretation, expose the uncertainty to the Narrator to handle gracefully, ask the user to decide, or fail gracefully. It cannot rewrite the past.

11.3 Correction Framework

- Factual correction — 'The chips were on the desk, not the table.' Updates canon state directly.
- Interpretive correction — 'He sounded nervous there, not angry.' Updates character or scene inference.
- Tonal correction — 'Make this softer and more intimate.' Adjusts Narrator generation policy.
- Intent correction — 'I want this to go in a more romantic direction.' Updates narrative steering variables.

12. Context Structure & Retrieval

The context system solves a fundamental problem: the LLM cannot receive everything it might need on every turn — token limits are finite and irrelevant detail degrades output quality. Instead the system uses a layered retrieval hierarchy. The LLM starts with a compact brief and requests additional depth through tool calls only when the current moment genuinely requires it.

This mirrors how a skilled dungeon master operates. They hold the entire world in their head but they do not read every character's full backstory at the start of every scene. They reach for specific detail when the moment calls for it — and that detail is already organised and ready to retrieve.

12.1 The Four-Layer Hierarchy

Context is organised into four retrieval layers. The LLM always receives Layer 0. It requests deeper layers via tool calls when needed. Multiple requests within the same layer are always batched into a single call to minimise round-trips.

Layer	Name	Contents	When used
0	Scene Brief	Current location name and sensory mood, characters present with one-line	Always — injected automatically every turn

		emotional state each, last 3–5 dialogue exchanges, player physical state, active narrative hooks	
1	Standard Detail	Full room contents list, character recent-memory summary, player inventory, full sensory detail of location, character relationship summary	Most non-trivial turns — single tool call batch
2	Deep Inspection	Specific object contents or state, one character's full relationship history with another, detailed emotional event log for a story-day, container inventory	Player interacts directly with specific objects or people
3	Archive Retrieval	Semantic search of memory database, historical event records, long-term biographical detail, off-screen event logs	Rare — significant narrative moments or direct historical queries

12.2 The Scene Brief — Layer 0 Specification

Layer 0 is the minimum viable context. It must be compact enough to leave room for generation — targeting 300–500 tokens — but informationally dense enough that the majority of turns complete without any additional retrieval calls. The Scene Brief contains:

- Location identifier and current sensory summary — One or two sentences describing the feel of the space right now: light, temperature, ambient sound, smell if notable.
- Characters present — Each active character listed with name, LOD level, and a single-line emotional state summary drawn from their volatile state layer.
- Recent exchange window — The last 3–5 turns of dialogue or action, verbatim. This is the only raw transcript that appears in the active context.
- Player state — Current physical state (fatigue, hunger, injury if any), emotional colour, and any items currently held or interacted with.
- Active narrative hooks — Story threads currently in motion that may need resolution, expressed as brief flags rather than full context.
- Player power mode — Whether the session is running simulation, collaborative, or author mode. This affects how the LLM handles player-proposed facts and corrections.

12.3 Batched Tool Calls

Every tool call the LLM makes adds a round-trip and therefore adds latency. To minimise this, all context requests within the same layer must be batched into a single call. The LLM submits a list of everything it needs — inspect table, get character emotional log, check inventory — and receives all results in one response. This is enforced by the tool call API design: there is no single-item endpoint at Layer 1 or Layer 2. Requests must be lists.

Performance target: For a simple dialogue turn, the LLM should complete using only Layer 0 with no additional calls. Layer 1 is acceptable for most interaction turns. Layer 2 and 3 calls should be rare enough that they are not noticeable in overall turn latency. The LLM generation call itself dominates at 5–15 seconds — database retrieval should add no more than a few hundred milliseconds.

12.4 Active Context Packet Assembly

At the start of each turn the World Agent assembles the active context packet. This is what the Narrative Engine receives. The assembly process:

- 1. Load the Scene Brief from cached scene state (always pre-built, never computed on-demand at turn time).
- 2. Run memory retrieval for each active character — vector similarity search returns top 5–8 memories pre-ranked.
- 3. Apply attentional filters from each character's volatile state to weight sensory detail.
- 4. Inject canonical flags for any unresolved details relevant to the current scene.
- 5. Append active narrative hooks and player power mode.
- 6. Deliver the assembled packet to the Narrative Engine.

Steps 2–5 are pre-computed asynchronously after the previous turn completes. By the time the player submits their next input, the context packet is already assembled and waiting. The only remaining latency is the LLM generation call itself.

13. Information Extractor

After every single turn — regardless of how mundane — the Information Extractor runs a lightweight pass over the completed exchange. Its sole job is to identify what changed, what was introduced, and what needs to be written to state. It submits a structured diff to the World Agent, which validates and commits each item.

This system closes the most common gap in AI story tools: things mentioned once and then silently forgotten because they never made it into structured state. The chat felt continuous but the world did not actually update. The Extractor prevents this entirely.

13.1 What the Extractor Tracks

The Extractor processes five entity categories in every pass. Like tool calls, all extraction findings are submitted as a single batched list — one commit operation to the World Agent, not one per entity.

- **People** — Any character mentioned by name or relationship descriptor gets a record check. A known character has their existing node updated with any new facts stated. An unknown character gets a new LOD 0 node created immediately, seeded with whatever facts were stated, and flagged for potential LOD promotion based on how much they are discussed.
- **Locations** — Any place mentioned gets noted. A new location creates a placeholder node with available facts. A known location has any newly stated detail appended. Adjacency or navigation information is logged as a relational edge.
- **Objects** — Any physical thing that moves, is interacted with, changes state, or is newly introduced gets logged. Movement is recorded with actor, time, and source location. New objects are created with inferred placement based on context.
- **Facts** — Anything stated as true about the world, a character, or a relationship is extracted and tagged by provenance. Did the player state this? Did a character reveal it? Did the system generate it? The source determines how much authority it carries.
- **Events** — Anything that happened, even minor, receives a lightweight event record. These seed the memory engine and can be retrieved later if they become contextually relevant.

13.2 Provenance Tagging on Extraction

Every item the Extractor writes to state is tagged with its source. This is critical because user-stated and directly observed facts carry much higher authority than inferred or system-generated ones. The provenance tag determines how easily a detail can be overwritten later and how it is weighted in memory retrieval scoring.

authored	Explicitly stated or established by the player. Highest authority. Cannot be altered by generation.
observed	Witnessed directly in scene by player or characters present. Locked once committed.
simulated	Generated by the Simulation Engine as part of off-screen behaviour. Stable but can be overridden by authored facts.
inferred	Logical deduction by the Extractor or engine. Weakest authority. Replaced whenever harder evidence arrives.
unresolved	Noted as a gap but not yet filled. Flagged for lazy resolution when first needed.

13.3 Extractor Timing & Cost

The Extractor runs asynchronously after each turn — it does not block the player from submitting their next input. It uses a small, fast model rather than the full Narrator model. Its prompt is strictly structured: read the exchange, produce a JSON diff, nothing else. The output is a machine-readable list, not prose. This keeps the per-turn extraction cost low and the latency invisible to the player.

14. Memory Compactor

The chat transcript cannot be held in active context indefinitely. Token limits are finite and very old raw dialogue is far less useful than a well-structured summary of what it contained. The Memory Compactor periodically processes older segments of the transcript, distils them into structured memory records, embeds them in the vector database, and retires the raw text from active context.

The goal is to make the story feel continuous even though the raw transcript is not always present. A well-compacted memory is more valuable than a raw transcript because it is semantically indexed, emotionally tagged, and retrievable by relevance rather than recency alone.

14.1 Compaction Triggers

Compaction is triggered by specific conditions, not by a fixed turn counter. Firing at arbitrary intervals risks breaking the texture of an active exchange.

- **Scene transition** — The most natural compaction point. When the player moves to a new location or time-skips, the previous scene is compacted as a unit.
- **Significant event** — A major world-initiated event or canonical turning point triggers compaction of the preceding segment, preserving the lead-up context as a structured record.
- **Sleep or time-skip in the story** — When the story advances through a rest period, all preceding content is a natural compaction candidate.
- **Transcript length threshold** — As a fallback, if the transcript reaches a defined length without a natural trigger, a compaction pass runs on the oldest N turns that are not part of the current active scene.

14.2 What a Compacted Memory Contains

The compactor produces a structured memory record for each segment it processes. This record becomes a permanent entry in the memory engine and is embedded for vector retrieval.

- **Actors involved and their roles in the segment**
- **Location(s) where events occurred**
- **Factual summary** — what happened, in order
- **Emotional summary** — how it felt, for whom, what shifted
- **Importance score** — derived from event impact and relationship significance

- Key details that must not be lost — objects introduced, facts stated, decisions made
- Unresolved threads flagged for follow-up

14.3 The Rolling Window

The active transcript window always retains the last N turns verbatim, because recent raw dialogue matters for tone and the precise texture of ongoing conversation. Compaction never touches this window. Only content older than the window boundary is eligible for compaction. This means the LLM always has immediate conversational context in raw form, and deeper history in structured retrievable form.

Design principle: The compactor should never be noticeable to the player. If a character said something three scenes ago and it becomes relevant now, the memory engine should surface it through retrieval — not because the full transcript is still present, but because a well-structured memory record captured it and the semantic search found it.

15. Consistency Checker

The Consistency Checker is a background process that runs at scene transitions — not every turn, but regularly enough to catch errors before they compound. Its job is to audit recent state writes and identify contradictions, then flag them for resolution before the next scene begins. It is the safety net that prevents extraction errors, generation drift, and edge-case bugs from accumulating into a world that no longer makes internal sense.

Without this process, small errors compound quietly. A character's location gets written incorrectly by an extraction error. Three turns later a second error builds on the first. By the time the player notices something is wrong, the contradictions are deeply embedded and difficult to unwind. The Consistency Checker catches errors at the first scene boundary after they are introduced.

15.1 What the Checker Validates

At each scene transition the Checker runs a structured audit across four domains:

- Location consistency — Is every character in a location that is reachable given their last known position and the time that has elapsed? Is every object in a location that is plausible given its last recorded movement? Are there any orphaned entities with no valid location?
- Temporal consistency — Do the timestamps on recent events form a valid chronological sequence? Has any event been logged with a time that precedes an event it logically

depends on? Have any physiological states (hunger, fatigue) been updated in a way that ignores elapsed time?

- Relational consistency — Do relationship edges between characters reflect what has actually occurred in the story? Has any emotion or attitude been written that directly contradicts an established canonical fact without a logged cause?
- Canon integrity — Have any inferred facts been written that conflict with canonical or authored facts? Has any detail been promoted to canonical without proper authority? Are there unresolved placeholders that have been referenced as if they were settled?

15.2 Resolution Protocol

When the Checker identifies a contradiction it classifies it by severity and routes it accordingly:

Minor	Silently resolved by the World Agent using the lower-authority fact as the error. Logged for review but does not interrupt play.
Moderate	Flagged as an unresolved conflict. The engine holds the contradiction in suspension and surfaces it to the Narrator as an ambiguity to handle gracefully in the next scene.
Severe	Presented to the player as a canon conflict requiring their decision. The engine offers two or three coherent resolutions and applies whichever the player selects.

The Checker does not attempt to resolve contradictions unilaterally at the moderate or severe level. Choosing between two conflicting canonical facts is not an engine decision — it is either a user decision or a graceful narrative reframing. The engine surfaces the problem cleanly and presents options. It does not quietly pick one and hope the player does not notice.

15.3 Checker Timing & Performance

The Checker runs as a background job triggered by scene transition events. It operates on a snapshot of state taken at the moment of transition, not on live state, so it does not block the player from entering the new scene. Results are ready and applied before the first turn of the new scene executes. On well-maintained state the Checker should find nothing and complete in milliseconds. The cost of running it is trivial compared to the cost of not running it.

16. Full Turn Pipeline

The following is the complete sequence of operations for a single player turn, from input to rendered output, including all background processes.

- 1. Player input received.

- 2. World Agent loads pre-assembled context packet (built asynchronously after previous turn).
- 3. Scope detection — classify the turn type: dialogue, physical action, scene transition, correction, or query.
- 4. Narrative Engine (LLM) receives context packet and generates proposed output, including any tool call requests for additional context.
- 5. Tool call batch processed — all requested Layer 1, 2, or 3 context retrieved in a single batched query and returned.
- 6. Narrative Engine completes generation with full context.
- 7. World Agent validates generated proposals against canon before any state is committed.
- 8. Validated state changes written to database. New entities, movements, and facts committed.
- 9. Narrator Agent renders final prose from validated state.
- 10. Output delivered to player.
- 11. [Async] Information Extractor runs on completed exchange, produces diff, submits to World Agent for commit.
- 12. [Async] Context packet for next turn pre-assembled and cached.
- 13. [Async, on trigger] Memory Compactor runs if scene transition or length threshold reached.
- 14. [Async, on trigger] Consistency Checker runs if scene transition reached.

Latency note: Steps 1–10 are the critical path and determine what the player waits for. Steps 11–14 run in the background and are invisible to the player. The LLM generation call (step 6) dominates latency at 5–15 seconds. All other steps on the critical path should contribute no more than a few hundred milliseconds in total on a properly indexed database.

17. Database Architecture

For a text-only narrative engine no game engine is required. Two database types cover all persistence needs, combined with an in-memory context assembly layer.

17.1 Relational Database (SQLite)

Handles all hard-structured truth — the deterministic backbone of the simulation. For solo or small-scale use, SQLite is sufficient and requires no external service. Core tables:

- locations — id, parent_id, name, sensory_state (JSON), last_visited, connection_graph (JSON).

- objects — id, current_location_id, surface_or_container, owner_id, movement_log (JSON), interactable_state, canon_status, narrative_flags.
- characters — id, lod_level, biographical_summary, speech_profile (JSON), physical_state (JSON), schedule (JSON).
- relationships — character_a_id, character_b_id, dimension (trust/affection/fear/etc.), value (float), last_updated, event_log (JSON).
- canon_log — fact_id, content, provenance_tag, timestamp, locked (boolean), source_turn_id.
- events — id, actors (JSON), locations (JSON), factual_summary, emotional_summary, importance, timestamp, tags (JSON), canon_status.
- narrative_hooks — id, thread_id, description, pressure_value, last_updated, status.

17.2 Vector Database (ChromaDB)

Handles memory retrieval. Every memory record is embedded as a vector at creation time using a small embedding model. Retrieval is a semantic similarity search against the current scene context — fast, relevance-aware, and not dependent on keyword matching. ChromaDB runs locally with no external service dependency, making it appropriate for personal or small-scale implementation.

17.3 Performance Strategy

- Pre-computation — Context packets are assembled after each turn, not before. By the time the player acts, the packet is ready.
- Lazy expansion — World detail is generated only when needed. A door is not detailed until opened. A character's history is not expanded until their LOD promotes.
- Event-driven state updates — Character emotional state is not recalculated from scratch each turn. Current state is stored and only delta-updated when an event triggers a change.
- Indexed queries — All hot-path database queries run against indexed columns. Location lookups by id, object lookups by location_id, character lookups by lod_level and scene_id should all return in single-digit milliseconds.

18. Player Interaction Model

The player is simultaneously a participant in the story and a co-author of it. The degree of authorial power is configurable and can shift across sessions.

18.1 Player Power Modes

- Simulation mode — The player acts within the world. Corrections limited to genuine misinterpretations. The world has strong resistance to improbable changes. World-initiated events fire at full frequency.

- Collaborative mode — Balanced control. The player can nudge hidden context and adjust likely outcomes. World-initiated events fire at moderate frequency.
- Author mode — Maximum narrative control. The player can declare many facts and the engine focuses on making them coherent. World-initiated events fire only when the player approves or when pressure is extreme.

18.2 Variable Story Setup

- Minimal — A short story description, a few paragraphs. The engine generates world scaffold, characters, and opening state from this alone.
- Guided — Key characters defined with relationships and a starting situation. Engine fills everything else.
- Full authored — Complete world map, character histories, family networks, active events, prior story. The engine runs it as specified.

The engine starts from whatever is given and generates only what is needed. Nothing is fully elaborated until it becomes relevant.

19. Recommended Build Order

This architecture is designed to scale but should be built incrementally. Each step is independently testable. The system is viable as an interactive story engine from step 4 onward.

- 1. SQLite schema — locations, objects, characters, relationships, canon_log, events.
- 2. Single scene proof of concept — one room, two characters, object persistence verified manually.
- 3. Manual context packet assembly — hardcoded first, then dynamic query-based.
- 4. Single Narrator LLM call with the Layer 0 context packet. No agents yet. Story works.
- 5. Tool call layer — implement Layer 1 batched retrieval. LLM can now request room detail and character summaries.
- 6. Information Extractor — async post-turn extraction. World begins updating from what is said.
- 7. ChromaDB memory layer — embedding and retrieval for episodic memory. Layer 3 retrieval unlocked.
- 8. Memory Compactor — scene-transition triggered. Long stories remain coherent.
- 9. Character Agent split — LOD 3 characters get their own generation calls.
- 10. World Agent and validation pipeline — proposals must pass before commit.
- 11. Simulation Engine — off-screen behaviour, schedules, physiological drift.

- 12. Random Event System — narrative pressure tracking, trigger categories, intensity scaling.
- 13. Consistency Checker — scene-transition audit. World integrity maintained at scale.

Final principle: You are not building a chatbot. You are building a world. The story output is just one interface over a persistent, reactive, coherent simulation. The weight of that simulation — the chips still on the table, the ex who calls at the worst moment, the crash that nobody chose — is what makes the story feel real.